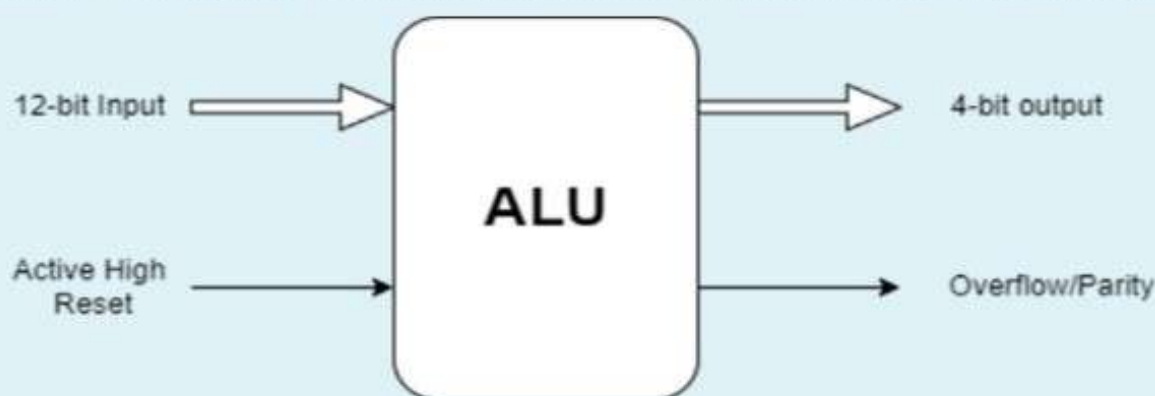


You are making your own microprocessor and today, you will design and implement a unique Arithmetic Logic Unit (ALU) for it. You decide to have minimal functions to start with and you also plan to make your own programming language to operate it. The functions of the ALU and code is given below:



The ALU has a 12-bit input bus, 1 reset input, a 4-bit output bus, and 1 overflow/pairty output.

When you want to instruct your ALU to perform any operation, you must supply it with an 'opcode' as input. For this application, you will be using a 12-bit opcode. Here is what each bit represents:

11	10	9	8	7	6	5	4	3	2	1	0
Mode		Parity	Direction	b				a			

- **opcode[11:10]** = 2-bit mode that

defines the operation that needs to be performed.

- ☐ 00 => addition
- ☐ 01 => subtraction
- ☐ 10 => multiplication

- **opcode[9]** = Parity bit. This bit defines if the parity that needs to be generated is even or odd.
 - 1 => even parity needs to be generated iff mode=11
 - 0 => odd parity needs to be generated iff mode=11
- **opcode[8]** = Direction bit for subtraction.
 - 1 => Perform a-b iff mode=01
 - 0 => Perform b-a iff mode=01
- **opcode[7:4]** = input b, a 4-bit binary number
- **opcode[3:0]** = input a, another 4-bit binary number

This 12-bit opcode is given as the 12-bit input of the ALU.

An 8-bit reg '**data**' is defined as the concatenation of a & b.

Ex: if a=1001 & b=0110, data=10010110

All the following operations must be implemented using INSTANTIATION as follows:

(Make separate modules separately using data flow and then call them.)

- **ADDER** (inputs *a* & *b*, and outputs *sum* & *overflow*),
- **SUBTRACTOR** (inputs *a* & *b*, and outputs *difference* & *overflow*),
- **MULTIPLIER** (inputs *a* & *b*, and outputs *product* & *overflow*),
- **PARITY_GENERATOR** (inputs *data* & *parity bit*, and outputs *generated_parity_bit*)

The 4-bit output bus is multiplexed and shared between all the outputs of the instantiated modules.

In the testbench, perform the following tasks on the ALU (MUST BE IN THE SAME ORDER):

- 1) $10+5 \Rightarrow b=1010 \ a=0101$
- 2) $15+5 \Rightarrow b=1111 \ a=0101$
- 3) $10-5$ with direction bit=1 $\Rightarrow b=1010 \ a=0101$
- 4) $10-5$ with direction bit=0 $\Rightarrow b=1010 \ a=0101$
- 5) $5-10$ with direction bit=0 $\Rightarrow b=0101 \ a=1010$
- 6) generate even parity for data=10100101
- 7) generate odd parity for data=10100101

Example: If you want to perform $2+3$,

Mode $\Rightarrow 00$

Parity $\Rightarrow$ don't care, assume 1

Direction $\Rightarrow$ don't care, assume 1

a $\Rightarrow$ binary representation of 2 = 0010

b $\Rightarrow$ binary representation of 3 = 0011

Opcode $\Rightarrow$ Opcode $\Rightarrow 001100110010$